# Data Analytics - Assignment 1

***Student Name***: *Silvana Paredes*

***Student Code***: *DAE10409*

***Course***: *DA-Blue*

## Part 1: SQL Queries [15 points]

### 1. Show all rows from the students table. (1 mark)

This is the most basic query because it only requires the table name and retrieves all its columns. In this case, we need to show all rows from the students table, with all the columns.

```
SELECT *
FROM students;
```

**Output**

| student_id | first_name | last_name | major | year_level |
|---|---|---|---|---|
| 1 | Alice | Brown | Computer Science | 2 |
| 2 | Alicia | Stone | Computer Science | 1 |
| 3 | Bob | Smith | Economics | 3 |
| 4 | Chloe | Davis | Physics | 1 |
| 5 | David | Wilson | Computer Science | 4 |
| 6 | Evan | Johnson | Mathematics | 2 |
| 7 | Eva | Jon | Mathematics | 2 |
| 8 | Al | O'Connor | Economics | 3 |
| 9 | Ali | Nguyen | Physics | 1 |
| 10 | Leo | Ng | Computer Science | 2 |
| 11 | Maya | Patel | Economics | 4 |
| 12 | Nina | Khan | Mathematics | 1 |
| 13 | Oscar | Lee | Physics | 3 |
| 14 | Priya | Sharma | Computer Science | 2 |
| 15 | Quinn | Taylor | Economics | 2 |
| 16 | Ravi | Mehta | Mathematics | 4 |
| 17 | Sofia | Martinez | Computer Science | 1 |
| 18 | Tom | Anderson | Physics | 2 |
| 19 | Uma | Singh | Economics | 1 |
| 20 | Victor | Hughes | Mathematics | 3 |

### 2. Find students whose first name starts with 'Ali'. (2 marks)

This query displays the rows of the students table whose first name starts with "Ali", using the LIKE operator. In this case, the LIKE operator checks that the first characters are "Ali", and % means that any sequence of characters can follow.

```sql
SELECT *
FROM students
WHERE first_name LIKE 'Ali%';
```

Output

| student_id | first_name | last_name | major | year_level |
|---|---|---|---|---|
| 1 | Alice | Brown | Computer Science | 2 |
| 2 | Alicia | Stone | Computer Science | 1 |
| 9 | Ali | Nguyen | Physics | 1 |

### 3. Show course_name and total enrolled students for courses with at least 2 enrollments. (2 marks)

This query retrieves the course names from the **courses table** and counts the number of enrollments for each course by joining it with the **enrollments table** through the **course_id** field, which is the common key between them. It groups the results by **course_id** to count enrollments per course and filters them to include only courses with at least two enrollments.

Input

Run SQL

```sql
SELECT
  course_name,
  count(enrollment_id) AS total_enrolled_students
FROM courses AS c
INNER JOIN enrollments AS e
  ON c.course_id = e.course_id
GROUP BY e.course_id
HAVING COUNT(e.enrollment_id) >= 2;
```

Output

| course_name | total_enrolled_students |
|---|---|
| Intro to Databases | 4 |
| Database Systems | 3 |
| Microeconomics | 5 |
| Advanced Microeconomics | 2 |
| Linear Algebra I | 4 |
| Quantum Mechanics | 4 |
| Data Structures | 3 |
| Machine Learning | 3 |

### 4. Show the number of students in each major. (1 mark)

This query selects the **major column** from the **students table** and counts the number of students in each major by grouping the results by the major column.

```
SELECT
  major,
  count(student_id) AS number_of_students
FROM students
GROUP BY major;
```

**Output**

| major | number_of_students |
|---|---|
| Computer Science | 6 |
| Economics | 5 |
| Mathematics | 5 |
| Physics | 4 |

### 5. List each student and the number of courses they're enrolled in (show 0 if none). (1 mark)

This query lists all students and gets the number of courses they are enrolled in. It uses a LEFT JOIN with the enrollments table to include students who are not enrolled in any courses. The results are ordered by student_id.

```
SELECT
  s.student_id,
  first_name,
  last_name,
  COUNT(enrollment_id) AS number_of_courses
FROM students AS s
LEFT JOIN enrollments AS e
  ON s.student_id = e.student_id
GROUP BY s.student_id
ORDER BY s.student_id;
```

**Output**

| student_id | first_name | last_name | number_of_courses |
|---|---|---|---|
| 1 | Alice | Brown | 2 |
| 2 | Alicia | Stone | 2 |
| 3 | Bob | Smith | 2 |
| 4 | Chloe | Davis | 1 |
| 5 | David | Wilson | 2 |
| 6 | Evan | Johnson | 1 |
| 7 | Eva | Jon | 0 |

| | | | |
|---|---|---|---|
| 8 | Al | O'Connor | 1 |
| 9 | Ali | Nguyen | 1 |
| 10 | Leo | Ng | 3 |
| 11 | Maya | Patel | 2 |
| 12 | Nina | Khan | 1 |
| 13 | Oscar | Lee | 2 |
| 14 | Priya | Sharma | 2 |
| 15 | Quinn | Taylor | 1 |
| 16 | Ravi | Mehta | 2 |
| 17 | Sofia | Martinez | 2 |
| 18 | Tom | Anderson | 2 |
| 19 | Uma | Singh | 1 |
| 20 | Victor | Hughes | 2 |

6. **Show students enrolled in any course whose name contains 'Database', with student first name, last name and course name. (2.5 marks)**

This query selects the first_name and last_name of students, and the course_name for courses that contain 'Database' in their name. It needs an INNER JOIN between students and enrollments, and another INNER JOIN with courses to match students to their courses.

```
SELECT
  first_name,
  last_name,
  course_name
FROM students AS s
INNER JOIN enrollments AS e
  ON s.student_id = e.student_id
INNER JOIN courses as c
  ON e.course_id = c.course_id
WHERE course_name LIKE '%Database%';
```

**Output**

| first_name | last_name | course_name |
|---|---|---|
| Alice | Brown | Intro to Databases |
| Alicia | Stone | Database Systems |
| David | Wilson | Intro to Databases |
| Leo | Ng | Intro to Databases |
| Leo | Ng | Database Systems |
| Priya | Sharma | Intro to Databases |
| Sofia | Martinez | Database Systems |

### 7. Show the average year_level (rounded to 2 decimals) and the number of students for each major. (1 mark)

This query calculates the average year_level (rounded to two decimals) and counts the number of students for each major. It groups the results by the major column to show the data per major.

```
SELECT
  major,
  ROUND(AVG(year_level), 2) AS average,
  COUNT(student_id) AS number_of_students
FROM students
GROUP BY major;
```

**Output**

| major | average | number_of_students |
|---|---|---|
| Computer Science | 2 | 6 |
| Economics | 2.6 | 5 |
| Mathematics | 2.4 | 5 |
| Physics | 1.75 | 4 |

### 8. Show students who are not enrolled in any course. (1.5 marks)

This query selects students who are not enrolled in any courses. It performs a **LEFT JOIN** between students and enrollments, groups the results by student_id.

This query was based on the fifth exercise. However, this one uses the **HAVING** clause to filter for students with a count of zero enrollments. Using **IS NULL** on the joined table didn't retrieve rows, so counting enrollments is more reliable in this case.

```
SELECT
  first_name,
  last_name,
  COUNT(enrollment_id) AS number_of_courses
FROM students AS s
LEFT JOIN enrollments AS e
  ON s.student_id = e.student_id
GROUP BY s.student_id
HAVING count(enrollment_id) = 0;
```

**Output**

| first_name | last_name | number_of_courses |
|---|---|---|
| Eva | Jon | 0 |

**9. Show courses (name and id) that have more enrollments than 'Data Structures'. Include the number of students enrolled. (3 marks)**

This query selects the **course_id, course_name**, and the number of students enrolled in each course. It uses an **INNER JOIN** between courses and enrollments, groups the results by course, and filters to include only courses that have more enrollments than the course named 'Data Structures'.

To obtain the total number of enrollments for the course named **'Data Structures',** it was necessary to join the enrollments and courses tables and counting the enrollments. This result was used in the HAVING clause to filter for courses that have more enrollments than 'Data Structures'.

```sql
SELECT
  c.course_id,
  course_name,
  count(enrollment_id) AS number_students_enrolled
FROM courses AS c
INNER JOIN enrollments AS e
  ON c.course_id = e.course_id
GROUP BY c.course_id, c.course_name
HAVING COUNT(e.enrollment_id)> (
  SELECT COUNT(*)
    FROM enrollments AS e2
    INNER JOIN courses AS c2
      ON e2.course_id = c2.course_id
    WHERE c2.course_name = 'Data Structures'
);
```

Output

| course_id | course_name | number_students_enrolled |
|---|---|---|
| 1 | Intro to Databases | 4 |
| 3 | Microeconomics | 5 |
| 5 | Linear Algebra I | 4 |
| 6 | Quantum Mechanics | 4 |

## Part 2: Python Programming [15 points]:

## 1. Loading the Dataset (1 mark)

a. Load the datasets **PlayerStatistics.csv** and **Games.csv** (attached to the Assignment on DISCO). **(0.5 mark)**

**Code:**

```
import pandas as pd
import matplotlib.pyplot as plt

#-----------------------------------------------------------------------
#1.Loading the Dataset
#a.Read the files
df_games = pd.read_csv("Games.csv", sep=",", low_memory=False)
df_players = pd.read_csv("PlayerStatistics.csv", sep=",", low_memory=False)
```

**Explanation:**

This code loads two CSV files using pandas. I included the parameter **low_memory=False** because the files are large, and this option helps pandas read the data more efficiently.

b. Display the first **10** rows of each dataset. **(0.5 mark)**

**Code:**

```
#b.Display the first 10 rows of each dataset
print(f'\nGames file:\n{df_games.head(10)}')
print(f'\nPlayers file:\n{df_players.head(10)}')
```

**Output:**

```
Games file:
     gameId             gameDate  ... gameSubLabel seriesGameNumber
0  42400407  2025-06-22 20:00:00  ...       Game 7              7.0
1  42400406  2025-06-19 20:30:00  ...       Game 6              6.0
2  42400405  2025-06-16 20:30:00  ...       Game 5              5.0
3  42400404  2025-06-13 20:30:00  ...       Game 4              4.0
4  42400403  2025-06-11 20:30:00  ...       Game 3              3.0
5  42400402  2025-06-08 20:00:00  ...       Game 2              2.0
6  42400401  2025-06-05 20:30:00  ...       Game 1              1.0
7  42400306  2025-05-31 20:00:00  ...       Game 6              6.0
8  42400305  2025-05-29 20:00:00  ...       Game 5              5.0
9  42400315  2025-05-28 20:30:00  ...       Game 5              5.0

[10 rows x 17 columns]
```

```
Players file:
  firstName            lastName  ...  turnovers  plusMinusPoints
0     James             Johnson  ...        0.0              0.0
1      T.J.           McConnell  ...        7.0            -12.0
2     Myles              Turner  ...        1.0            -12.0
3    Pascal              Siakam  ...        1.0             -4.0
4      Alex              Caruso  ...        0.0              8.0
5    Isaiah         Hartenstein  ...        2.0              1.0
6      Tony             Bradley  ...        1.0             -2.0
7    Thomas              Bryant  ...        0.0              2.0
8      Shai  Gilgeous-Alexander  ...        1.0             11.0
9   Kenrich            Williams  ...        0.0             -1.0

[10 rows x 35 columns]
```

**Explanation:**

This code displays the first ten rows of each file. By default, the parameter is set to five:

## 2. Data Manipulation (3 marks)

a. Check for missing values in **both** datasets. **(1 mark)**

**Code:**

```
#2.Data Manipulation
#a.Check for missing values
print('\nCheck for missing values:')
print(f'\nGames file:\n{df_games.isnull().sum()}')
print(f'\nPlayers file:\n{df_players.isnull().sum()}')
```

**Output:**

```
Check for missing values:

Games file:
gameId                 0
gameDate               0
hometeamCity           0
hometeamName           0
hometeamId             0
awayteamCity           0
awayteamName           0
awayteamId             0
homeScore              0
awayScore              0
winner                 0
gameType               0
attendance          5029
arenaId                0
gameLabel          68087
gameSubLabel       71721
seriesGameNumber   66141
dtype: int64
```

```
Players file:
firstName                       0
lastName                        0
personId                        0
gameId                          0
gameDate                        0
playerteamCity                  0
playerteamName                  0
opponentteamCity                0
opponentteamName                0
gameType                        0
gameLabel                 1536855
gameSubLabel              1623051
seriesGameNumber          1492390
win                             0
home                            0
numMinutes                 161558
points                       1219
assists                      1219
blocks                       1219
steals                       1219
fieldGoalsAttempted          1219
fieldGoalsMade               1219
fieldGoalsPercentage         1219
threePointersAttempted       1219
threePointersMade            1219
threePointersPercentage      1219
freeThrowsAttempted          1219
freeThrowsMade               1219
freeThrowsPercentage         1219
reboundsDefensive            1219
reboundsOffensive            1219
reboundsTotal                1219
foulsPersonal                1219
turnovers                    1219
plusMinusPoints              1219
dtype: int64
```

**Explanation:**

This code checks for missing values in each DataFrame by counting the null values in each column. We can see that some columns contain null values in both files:

b. Create a new column in **PlayerStatistics.csv** called Full Name to combine both firstName and lastName field e.g. *James Johnson*. **(1 mark)**

**Code:**

```
#b.Create a new column in PlayerStatistics.csv called Full Name to combine both firstName and lastName field
df_players['fullName'] = df_players['firstName'] + ' ' + df_players['lastName']
print(f"\nFull name column:\n{df_players['fullName'].head(10)}")
```

**Output:**

```
Full name column:
0              James Johnson
1             T.J. McConnell
2               Myles Turner
3              Pascal Siakam
4                Alex Caruso
5         Isaiah Hartenstein
6               Tony Bradley
7              Thomas Bryant
8    Shai Gilgeous-Alexander
9           Kenrich Williams
Name: fullName, dtype: object
```

**Explanation:**

This code created a new column called Full Name by combining first and last name. Then, it prints the result to verify that it works correctly.

c. Identify the types of games the players played using the gameType field in **PlayerStatistics.csv,** subsequently change "Preseason" to "Pre-season" **(1 mark)**

**Code:**

```
#c.Identify the types of games the players played using the gameType field in PlayerStatistics.csv
game_types = df_players['gameType'].unique()
print(f'\nTypes of games:\n{game_types}')

#Replace 'Preseason' to 'Pre-season'
df_players['gameType'] = df_players['gameType'].replace('Preseason', 'Pre-season')
#Display the column after applying the changes
df_preseason = df_players.loc[df_players['gameType'] == 'Pre-season', 'gameType']
print(f'\nUpdated gameType column:\n{df_preseason}')
```

**Output:**

```
Types of games:
['Playoffs' 'Play-in Tournament' 'Regular Season' 'Preseason' 'NBA Cup'
 'NBA Emirates Cup']

Updated gameType column:
35091     Pre-season
35092     Pre-season
35093     Pre-season
35094     Pre-season
35095     Pre-season
             ...
765119    Pre-season
765120    Pre-season
765121    Pre-season
765122    Pre-season
765123    Pre-season
Name: gameType, Length: 54580, dtype: object
```

**Explanation:**

This code displays the different types of games in the file without repetitions using the unique() function. Then, it replaces all occurrences of the string 'Preseason' with 'Pre-season' and displays the column to verify the changes.

### 3. Data Summary (4 marks)

**Using Games.csv**

a. Identify and list categorical columns in **Games.csv**. **(1 mark)**

**Code:**

```
#3.Data Summary
#Using Games.csv
#a.Identify and list categorical columns in Games.csv.
df_cat = df_games.select_dtypes(include=['object','category']).columns
print(f'\nCategorical columns:\n{df_cat}')
```

**Output:**

```
Categorical columns:
Index(['gameDate', 'hometeamCity', 'hometeamName', 'awayteamCity',
       'awayteamName', 'gameType', 'gameLabel', 'gameSubLabel'],
      dtype='object')
```

**Explanation:**

This code displays the categorical columns in the Games file. These columns contain data that represent categories or groups rather than numerical values:

b. List the distinct/unique hometeamCity and count them in **Games.csv. (1 mark)**

**Code:**

```
#b.List the distinct/unique hometeamCity and count them in Games.csv.
#List
df_hometeamCity = df_games['hometeamCity'].unique()
print(f'\nUnique Hometeam Cities:\n{df_hometeamCity}')

#Count
num_hometeamCity = df_games['hometeamCity'].nunique()
print('\nNumber of Hometeam Cities:', num_hometeamCity)
```

**Output:**

```
Unique Hometeam Cities:
['Oklahoma City' 'Indiana' 'New York' 'Minnesota' 'Denver' 'Boston'
 'Cleveland' 'Golden State' 'Houston' 'Los Angeles' 'Detroit' 'Miami'
 'Milwaukee' 'Orlando' 'Memphis' 'Atlanta' 'Sacramento' 'Chicago'
 'New Orleans' 'San Antonio' 'Portland' 'Brooklyn' 'Philadelphia'
 'Phoenix' 'Utah' 'Dallas' 'Toronto' 'Washington' 'Charlotte' 'New Jersey'
 'Seattle' 'Vancouver' 'Kansas City' 'San Diego' 'Buffalo'
 'Kansas City-Omaha' 'Capital' 'Baltimore' 'Cincinnati' 'San Francisco'
 'St. Louis' 'Syracuse' 'Minneapolis' 'Ft. Wayne Zollner' 'Rochester'
 'Tri-Cities']

Number of Hometeam Cities: 46
```

**Explanation:**

This code displays the unique values in the **homeTeamCity** column and then counts how many there are.

c. Generate descriptive statistics for numerical features in **Games.csv. (1 mark)**

**Code:**

```
#c.Generate descriptive statistics for numerical features in Games.csv.
print('\nDescriptive statistics for numerical features in Games:\n',df_games.describe(include='number'))
```

**Output:**

```
Descriptive statistics for numerical features in Games:
              gameId    hometeamId  ...       arenaId  seriesGameNumber
count  7.187900e+04  7.187900e+04  ...  7.187900e+04       5738.000000
mean   2.583156e+07  1.610613e+09  ...  3.568417e+04          2.496340
std    6.420623e+06  8.660615e+00  ...  1.852905e+05          2.017287
min    1.030000e+07  1.610613e+09  ...  0.000000e+00          0.000000
25%    2.130058e+07  1.610613e+09  ...  5.000000e+00          1.000000
50%    2.630022e+07  1.610613e+09  ...  4.400000e+01          2.000000
75%    2.870060e+07  1.610613e+09  ...  1.350000e+02          4.000000
max    6.240000e+07  1.610613e+09  ...  1.000145e+06          7.000000

[8 rows x 9 columns]
```

**Explanation:**

This code prints the descriptive statistics (such as count, mean, standard deviation, minimum, quartiles, and maximum) for all the numerical columns in the Games DataFrame:

**Using PlayerStatistics.csv**

d. Identity the clubs the players play for, how many clubs do we have? **(1 mark)**

**Code:**

```
#Using PlayerStatistics.csv
#d.Identify the clubs the players play for, how many clubs do we have?
#List
df_clubs = df_players['playerteamName'].unique()
print(f'\nClubs:\n{df_clubs}')

#Count
num_clubs = df_players['playerteamName'].nunique()
print('\nNumber of clubs:', num_clubs)
```

**Output:**

```
Clubs:
['Pacers' 'Thunder' 'Knicks' 'Timberwolves' 'Nuggets' 'Celtics' 'Warriors'
 'Cavaliers' 'Rockets' 'Clippers' 'Pistons' 'Lakers' 'Magic' 'Bucks'
 'Heat' 'Grizzlies' 'Mavericks' 'Hawks' 'Kings' 'Bulls' 'Jazz' 'Pelicans'
 'Spurs' 'Raptors' 'Trail Blazers' 'Suns' 'Hornets' 'Nets' 'Wizards'
 '76ers' 'Bobcats' 'SuperSonics' 'Bullets' 'Braves' 'Royals' 'Nationals'
 'Zephyrs' 'Packers' 'Blackhawks']

Number of clubs: 39
```

**Explanation:**

This code displays the unique values in the **playerteamName** column and then counts how many there are (without repetitions):

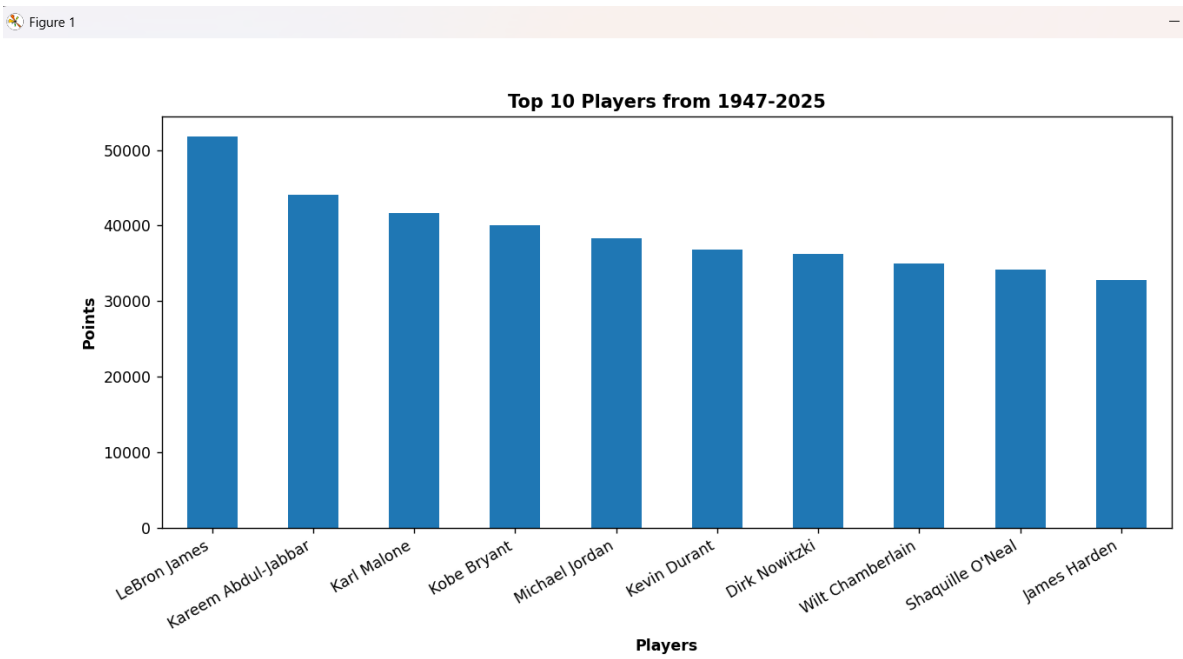
## 4. Players Analysis (4 marks)

a. Using a bar chart, Display the top 10 players from 1947-2025 based on points. **(0.5 mark)**

## Code:

```
#4.Players Analysis
#a.Using a bar chart, Display the top 10 players from 1947-2025 based on points.
#Get the top 10 players based on total points
df_top10 = df_players.groupby('fullName')['points'].sum().sort_values(ascending=False).head(10)
print(f'\nTop 10 Players from 1947-2025:\n{df_top10}')

#Bar chart
plt.figure(figsize=(12,6))
df_top10.plot(kind='bar')
plt.xlabel('Players', fontweight='bold')
plt.ylabel('Points', fontweight='bold')
plt.title('Top 10 Players from 1947-2025', fontweight='bold')
plt.xticks(rotation=30, ha='right')  # rotate the player names
plt.subplots_adjust(bottom=0.25)     # increase space at the bottom
plt.show()
```

## Output:




```
Top 10 Players from 1947-2025:
fullName
LeBron James           51873.0
Kareem Abdul-Jabbar    44149.0
Karl Malone            41689.0
Kobe Bryant            40100.0
Michael Jordan         38279.0
Kevin Durant           36855.0
Dirk Nowitzki          36212.0
Wilt Chamberlain       35024.0
Shaquille O'Neal       34221.0
James Harden           32770.0
Name: points, dtype: float64
```

## Explanation:

This code retrieves the top 10 players from 1947 to 2025 (all rows) based on the points column. It groups the data by fullName and sums the points for each player. Then, the results are sorted in descending order, and the first ten rows are selected. I chose the fullName column instead of personId because it allows me to display the players' names in the chart.

To create the chart, I used Matplotlib. Since the player names are long, I rotated the labels and adjusted the bottom spacing to improve the overall presentation.

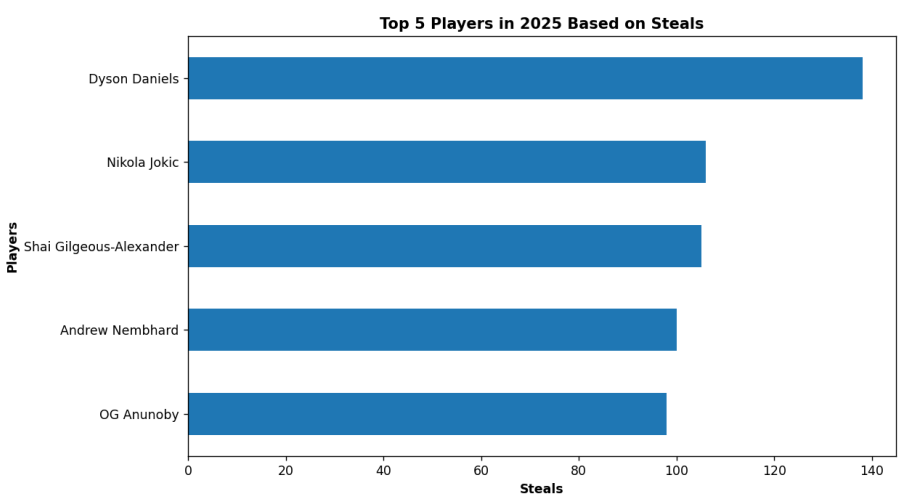
b. Visualize the top 5 players in 2025 based on steals using a chart of your choice. **(0.5 mark)**

**Code:**

```
#b.Visualize the top 5 players in 2025 based on steals using a chart of your choice.
#Top 5
df_players['gameDate'] = pd.to_datetime(df_players['gameDate'], errors='coerce')  #convert to a datetime format
df_players['year'] = df_players['gameDate'].dt.year                               #create a new column named year
df_2025 = df_players.loc[df_players['year'] == 2025]                              #filter by year
df_top5 = df_2025.groupby('fullName')['steals'].sum().sort_values(ascending=False).head(5)
print(f'\nTop 5 Players in 2025 Based on Steals:\n{df_top5}')

#Horizontal bar chart
plt.figure(figsize=(12,6))
df_top5.plot(kind='barh')
plt.xlabel('Steals', fontweight='bold')
plt.ylabel('Players', fontweight='bold')
plt.title('Top 5 Players in 2025 Based on Steals', fontweight='bold')
plt.subplots_adjust(left=0.25)       # increase space on the left
plt.gca().invert_yaxis()             # show the highest score at the top
plt.show()
```

**Output:**

```
Top 5 Players in 2025 Based on Steals:
fullName
Dyson Daniels              138.0
Nikola Jokic               106.0
Shai Gilgeous-Alexander    105.0
Andrew Nembhard            100.0
OG Anunoby                  98.0
Name: steals, dtype: float64
```




**Explanation:**

Following the steps from the previous exercise, I retrieved the top 5 players based on the steals column. Moreover, the rows were filtered for the year 2025.

Previously, I created a year column by extracting it from the gameDate column, which had to be converted to datetime type. This new column will be useful for the next exercises and simplifies filtering by year.

For the chart, I used a horizontal bar plot. I wanted to display the best player at the top, so I used invert_yaxis() to achieve this, because by default the top player appears at the bottom.

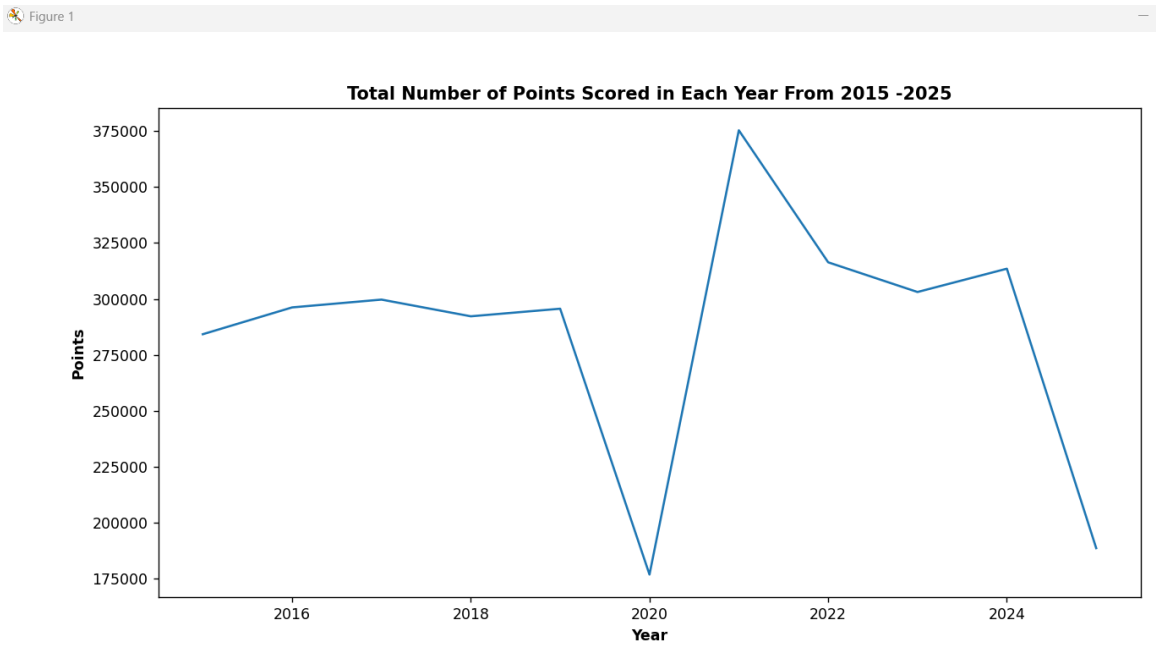
c. Using a line chat, display the total number of points scored in each year from 2015 -2025 **(1 mark)**

**Code:**

```
#c.Using a line chart, display the total number of points scored in each year from 2015 -2025
#Calculate the total number of points per year
df_years = df_players.loc[                                #filter by range of years
             (df_players['year'] >= 2015) &
             (df_players['year'] <= 2025)
             ]
df_points_year = df_years.groupby('year')['points'].sum()
print(f'\nTotal Number of Points Scored in Each Year From 2015 -2025:\n{df_points_year}')

#Line chart
plt.figure(figsize=(12,6))
df_points_year.plot(kind='line')
plt.xlabel('Year', fontweight='bold')
plt.ylabel('Points', fontweight='bold')
plt.title('Total Number of Points Scored in Each Year From 2015 -2025', fontweight='bold')
plt.show()
```

**Output:**

```
Total Number of Points Scored in Each Year From 2015 -2025:
year
2015    284264.0
2016    296243.0
2017    299730.0
2018    292271.0
2019    295658.0
2020    176976.0
2021    375325.0
2022    316375.0
2023    303119.0
2024    313525.0
2025    188761.0
Name: points, dtype: float64
```




**Explanation:**

In this exercise, the goal is to display the total number of points for each year from 2015 to 2025. The year column created in the previous exercise is useful for filtering the data within this range.

Next, I grouped the data by year and summed the total points scored.

Finally, I created a line chart using Matplotlib to visualize the total points for the selected years.

d. Canadian, Shai **Gilgeous-Alexander** was the MVP for 2025, display his total points for 2025 **(2 marks)**

**Code:**

```
#d.Canadian, Shai Gilgeous-Alexander was the MVP for 2025, display his total points for 2025
df_mvp = df_players.loc[
               (df_players['personId'] == 1628983) &              #filter by player
               (df_players['year'] == 2025)                       #filter by year
                   ]
total_mvp = df_mvp['points'].sum()
print('\nTotal points for Shai Gilgeous-Alexander in 2025:', total_mvp)
```

**Output:**

```
Total points for Shai Gilgeous-Alexander in 2025: 2172.0
```

**Explanation:**

This code calculates the total points scored in 2025 by the player Shai Gilgeous-Alexander. I filtered by personId to avoid any confusion due to possible misspellings in the name. Then, I grouped the data by year (2025) to sum the total points. In this case, the result is a single number.

### 5. Games Analysis (3 marks)

a. Display in a dataframe, the top 3 games with the highest attendance, include the hometeamName and awayteamName. **(1 mark)**

**Code:**

```
#5.Games Analysis
#a.Display in a dataframe, the top 3 games with the highest attendance,
#include the hometeamName and awayteamName.
df_top3 = df_games.sort_values(by='attendance', ascending=False).head(3)
df_top3 = df_top3[['hometeamName', 'awayteamName', 'attendance']]
print(f'\nTop 3 Games with the Highest Attendance:\n{df_top3}')
```

**Output:**

```
Top 3 Games with the Highest Attendance:
       hometeamName awayteamName  attendance
40087          Nets         Jazz    200049.0
39871  Timberwolves         Jazz    144442.0
3453          Spurs     Warriors     68323.0
```

**Explanation:**

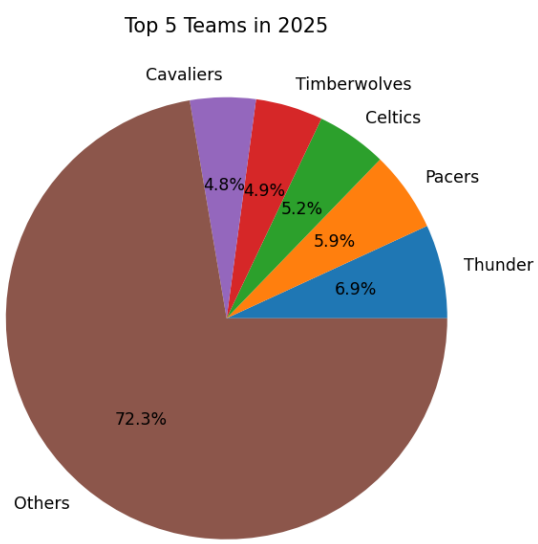
This code creates a DataFrame with the top 3 games by attendance. It sorts the **attendance** column in descending order, selects the first three rows, and then keeps only the homeTeamName, awayTeamName, and attendance columns for display.

b. Using a pie chart display the winning percentage of the top 5 teams in 2025 (the rest should be considered as "**Others**" – meaning the pie chart should have **6 pies**). **(2 mark)**

**Code:**

```
#b.Using a pie chart display the winning percentage of the top 5 teams in 2025
#(the rest should be considered as "Others" - meaning the pie chart should have 6 pies)
df_games['gameDate'] = pd.to_datetime(df_games['gameDate'], errors='coerce')    #convert to a datetime format
df_games['year'] = df_games['gameDate'].dt.year                                 #create a new column named year

#Filter by year 2025
df_2025 = df_games[df_games['year'] == 2025]

#Create a dictionary for the team names which played in 2025
df_teams = df_2025[['hometeamId','hometeamName']].drop_duplicates().sort_values('hometeamId')
print(f'\nTeam names:\n{df_teams}')

#Count the number of wins and get the top 5
df_top5teams_all = df_2025['winner'].value_counts()
df_top5teams = df_top5teams_all.head(5).reset_index()
df_top5teams.columns = ['hometeamId', 'wins']                               #rename columns

others_wins = df_top5teams_all.iloc[5:].sum()                               #calculate 'Others' group
df_top5teams = df_top5teams.merge(df_teams, on='hometeamId', how='left')    #merge the winner IDs with the corresponding team names

#Add Others group to the final dataframe
df_others = pd.DataFrame({                                                  #create 'Others' line in a dataframe
    'hometeamId': [None],
    'wins': [float(others_wins)],
    'hometeamName': ['Others']
})

df_top5teams = pd.concat([df_top5teams, df_others], ignore_index=True)      #add Others group to the final dataframe
print(f'\nTop 5 Teams in 2025:\n{df_top5teams}')


#Create the pie
plt.figure(figsize=(12,6))
plt.pie(df_top5teams['wins'], labels=df_top5teams['hometeamName'], autopct='%1.1f%%')
plt.title('Top 5 Teams in 2025')
plt.show()
```

**Output:**



```
Team names:
     hometeamId   hometeamName
85   1610612737          Hawks
22   1610612738        Celtics
24   1610612739      Cavaliers
93   1610612740       Pelicans
87   1610612741          Bulls
112  1610612742      Mavericks
20   1610612743        Nuggets
25   1610612744       Warriors
40   1610612745        Rockets
44   1610612746       Clippers
46   1610612747         Lakers
53   1610612748           Heat
54   1610612749          Bucks
11   1610612750   Timberwolves
100  1610612751           Nets
8    1610612752         Knicks
55   1610612753          Magic
1    1610612754         Pacers
103  1610612755          76ers
106  1610612756           Suns
96   1610612757  Trail Blazers
86   1610612758          Kings
94   1610612759          Spurs
0    1610612760        Thunder
131  1610612761        Raptors
109  1610612762           Jazz
60   1610612763      Grizzlies
134  1610612764        Wizards
45   1610612765        Pistons
141  1610612766        Hornets
```

```
Top 5 Teams in 2025:
   hometeamId   wins  hometeamName
0  1610612760   57.0       Thunder
1  1610612754   49.0        Pacers
2  1610612738   43.0       Celtics
3  1610612750   41.0  Timberwolves
4  1610612739   40.0     Cavaliers
5        None  601.0        Others
```

**Explanation:**

This code creates a pie chart showing the winning percentages of the top 5 teams in 2025, with the rest grouped as "Others," resulting in 6 segments.

First, a year column is created from the **gameDate** column (converted to datetime) to filter the games for 2025 (as in previous exercises).

The winner column contains **team IDs**, so we count the number of wins for each team and sort them to get the top 5. The "Others" group is calculated as the sum of wins from the sixth team to the last.

To display team names instead of IDs, a dictionary of team IDs and names was previously created, and then, merged with the top 5 teams (before adding Others). The "Others" row is then added to the DataFrame.

The pie chart is generated using Matplotlib. The labels parameter shows team names, and the autopct='%1.1f%%' parameter calculates and displays the winning percentage of each segment based on the wins values.

**Note**: If we analyzed multiple years, some team IDs have different names due to commercial name changes. In 2025, this is not an issue. Handling multiple years would require an additional step to get the latest team name based on dates.

### Final Note:

In the print statements, I used **f-strings** because when displaying a description and a DataFrame in the same line, concatenation caused an extra space to appear on the first row of the DataFrame output. Using f-strings avoids this formatting issue, keeps the output aligned, and allows writing the text and DataFrame on the same line.

## Appendix: Source Code

```
"""
Assignment 1
Part 2: Python Programming
```

In this assignment, you will analyze two datasets; PlayerStatistics.csv and Games.csv.
These datasets contain player statistics for NBA (National Basket Association) games played from 1947-2025.
The datasets provide an avenue to research basketball history, players' performance, games data and the teams' performance.

Student Name: Silvana Paredes
Date: 03/10/2025
"""

```
import pandas as pd
import matplotlib.pyplot as plt

#-----------------------------------------------------------------------
#1.Loading the Dataset
#a.Read the files
df_games = pd.read_csv("Games.csv", sep=",", low_memory=False)
df_players = pd.read_csv("PlayerStatistics.csv", sep=",", low_memory=False)

#b.Display the first 10 rows of each dataset
print(f'\nGames file:\n{df_games.head(10)}')
print(f'\nPlayers file:\n{df_players.head(10)}')

#-----------------------------------------------------------------------
#2.Data Manipulation
#a.Check for missing values
print('\nCheck for missing values:')
print(f'\nGames file:\n{df_games.isnull().sum()}')
print(f'\nPlayers file:\n{df_players.isnull().sum()}')

#b.Create a new column in PlayerStatistics.csv called Full Name to combine both firstName and lastName field
df_players['fullName'] = df_players['firstName'] + ' ' + df_players['lastName']
print(f"\nFull name column:\n{df_players['fullName'].head(10)}")

#c.Identify the types of games the players played using the gameType field in PlayerStatistics.csv
game_types = df_players['gameType'].unique()
print(f'\nTypes of games:\n{game_types}')


#Replace 'Preseason' to 'Pre-season'
df_players['gameType'] = df_players['gameType'].replace('Preseason', 'Pre-season')
#Display the column after applying the changes
df_preseason = df_players.loc[df_players['gameType'] == 'Pre-season', 'gameType']
print(f'\nUpdated gameType column:\n{df_preseason}')

#-----------------------------------------------------------------------
#3.Data Summary
#Using Games.csv
#a.Identify and list categorical columns in Games.csv.
```

```
df_cat = df_games.select_dtypes(include=['object','category']).columns
print(f'\nCategorical columns:\n{df_cat}')

#b.List the distinct/unique hometeamCity and count them in Games.csv.
#List
df_hometeamCity = df_games['hometeamCity'].unique()
print(f'\nUnique Hometeam Cities:\n{df_hometeamCity}')

#Count
num_hometeamCity = df_games['hometeamCity'].nunique()
print('\nNumber of Hometeam Cities:', num_hometeamCity)

#c.Generate descriptive statistics for numerical features in Games.csv.
print('\nDescriptive statistics for numerical features in Games:\n',df_games.describe(include='number'))

#Using PlayerStatistics.csv
#d.Identify the clubs the players play for, how many clubs do we have?
#List
df_clubs = df_players['playerteamName'].unique()
print(f'\nClubs:\n{df_clubs}')

#Count
num_clubs = df_players['playerteamName'].nunique()
print('\nNumber of clubs:', num_clubs)

#---------------------------------------------------------------------------
#4.Players Analysis
#a.Using a bar chart, Display the top 10 players from 1947-2025 based on points.
#Get the top 10 players based on total points
df_top10 = df_players.groupby('fullName')['points'].sum().sort_values(ascending=False).head(10)
print(f'\nTop 10 Players from 1947-2025:\n{df_top10}')

#Bar chart
plt.figure(figsize=(12,6))
df_top10.plot(kind='bar')
plt.xlabel('Players', fontweight='bold')
plt.ylabel('Points', fontweight='bold')
plt.title('Top 10 Players from 1947-2025', fontweight='bold')
plt.xticks(rotation=30, ha='right')  # rotate the player names
plt.subplots_adjust(bottom=0.25)     # increase space at the bottom
plt.show()

#b.Visualize the top 5 players in 2025 based on steals using a chart of your choice.
#Top 5
df_players['gameDate'] = pd.to_datetime(df_players['gameDate'], errors='coerce')  #convert to a datetime
format
df_players['year'] = df_players['gameDate'].dt.year                          #create a new column named year
df_2025 = df_players.loc[df_players['year'] == 2025]                          #filter by year
df_top5 = df_2025.groupby('fullName')['steals'].sum().sort_values(ascending=False).head(5)
print(f'\nTop 5 Players in 2025 Based on Steals:\n{df_top5}')
```

```
#Horizontal bar chart
plt.figure(figsize=(12,6))
df_top5.plot(kind='barh')
plt.xlabel('Steals', fontweight='bold')
plt.ylabel('Players', fontweight='bold')
plt.title('Top 5 Players in 2025 Based on Steals', fontweight='bold')
plt.subplots_adjust(left=0.25)      # increase space on the left
plt.gca().invert_yaxis()            # show the highest score at the top
plt.show()

#c.Using a line chart, display the total number of points scored in each year from 2015 -2025
#Calculate the total number of points per year
df_years = df_players.loc[                          #filter by range of years
          (df_players['year'] >= 2015) &
          (df_players['year'] <= 2025)
          ]
df_points_year = df_years.groupby('year')['points'].sum()
print(f'\nTotal Number of Points Scored in Each Year From 2015 -2025:\n{df_points_year}')

#Line chart
plt.figure(figsize=(12,6))
df_points_year.plot(kind='line')
plt.xlabel('Year', fontweight='bold')
plt.ylabel('Points', fontweight='bold')
plt.title('Total Number of Points Scored in Each Year From 2015 -2025', fontweight='bold')
plt.show()

#d.Canadian, Shai Gilgeous-Alexander was the MVP for 2025, display his total points for 2025
df_mvp = df_players.loc[
             (df_players['personId'] == 1628983) &            #filter by player
             (df_players['year'] == 2025)                     #filter by year
                ]
total_mvp = df_mvp['points'].sum()
print('\nTotal points for Shai Gilgeous-Alexander in 2025:', total_mvp)

#-----------------------------------------------------------------------------
#5.Games Analysis
#a.Display in a dataframe, the top 3 games with the highest attendance,
#include the hometeamName and awayteamName.
df_top3 = df_games.sort_values(by='attendance', ascending=False).head(3)
df_top3 = df_top3[['hometeamName', 'awayteamName', 'attendance']]
print(f'\nTop 3 Games with the Highest Attendance:\n{df_top3}')

#b.Using a pie chart display the winning percentage of the top 5 teams in 2025
#(the rest should be considered as “Others” – meaning the pie chart should have 6 pies)
df_games['gameDate'] = pd.to_datetime(df_games['gameDate'], errors='coerce')    #convert to a datetime
format
df_games['year'] = df_games['gameDate'].dt.year                          #create a new column named year

#Filter by year 2025
df_2025 = df_games[df_games['year'] == 2025]
```

```
#Create a dictionary for the team names which played in 2025
df_teams = df_2025[['hometeamId','hometeamName']].drop_duplicates().sort_values('hometeamId')
print(f'\nTeam names:\n{df_teams}')

#Count the number of wins and get the top 5
df_top5teams_all = df_2025['winner'].value_counts()
df_top5teams = df_top5teams_all.head(5).reset_index()
df_top5teams.columns = ['hometeamId', 'wins']                          #rename columns

others_wins = df_top5teams_all.iloc[5:].sum()                          #calculate 'Others' group
df_top5teams = df_top5teams.merge(df_teams, on='hometeamId', how='left')    #merge the winner IDs
with the corresponding team names

#Add Others group to the final dataframe
df_others = pd.DataFrame({                                             #create 'Others' line in a dataframe
    'hometeamId': [None],
    'wins': [float(others_wins)],
    'hometeamName': ['Others']
})

df_top5teams = pd.concat([df_top5teams, df_others], ignore_index=True)      #add Others group to the
final dataframe
print(f'\nTop 5 Teams in 2025:\n{df_top5teams}')

#Create the pie
plt.figure(figsize=(12,6))
plt.pie(df_top5teams['wins'], labels=df_top5teams['hometeamName'], autopct='%1.1f%%')
plt.title('Top 5 Teams in 2025')
plt.show()
```